

Atividade AVL

Eduardo Lemes CT3037568
Guilherme Batista de Souza CT3037274

Turma: ADS – Estrutura de Dados 2

Data: 02/11/2025

Parte 2:

LL

$A(-2)$
 \
 $B(-1)$
 \
 C

$\Rightarrow$

B
 /
 A \
 C

$\left\{ \begin{array}{l} FB(n) = -2 \text{ e} \\ FB(n, dir) \leq 0 \end{array} \right.$

RR

$A(+2)$
 /
 $B(+1)$
 /
 C

$\Rightarrow$

B
 /
 C \
 A

$\left\{ \begin{array}{l} FB(n) = +2 \text{ e} \\ FB(n, esq) \geq 0 \end{array} \right.$

LR

$A(+2)$
 /
 $B(-1)$
 \
 C

$\Rightarrow$

A
 /
 C
 /
 B

$\Rightarrow$

C
 /
 B \
 A

$\left\{ \begin{array}{l} FB(n) = +2 \text{ e} \\ FB(n, esq) < 0 \end{array} \right.$

RL

$A(-2)$
 \
 $B(+1)$
 /
 C

$\Rightarrow$

A
 \
 C
 \
 B

$\Rightarrow$

C
 /
 A \
 B

$\left\{ \begin{array}{l} FB(n) = -2 \text{ e} \\ FB(n, dir) > 0 \end{array} \right.$

tilibra

Adelson-Velsky e Landis

Georgy Adelson-Velsky e Evgenii Landis foram matemáticos e cientistas da computação soviéticos que, em 1962, inventaram a árvore AVL, a primeira estrutura de dados de árvore de busca binária auto-balanceada.

As árvores AVL foram criadas no início da década de 1960, um período de grande crescimento e pesquisa na área de ciência da computação, especialmente na União Soviética.

Antes das árvores AVL, as árvores de busca binária comuns podiam degenerar em estruturas semelhantes a listas ligadas, tornando as operações de busca, inserção e remoção ineficientes. A inovação de Adelson-Velsky e Landis foi criar uma estrutura de dados que mantinha o balanceamento de forma automática. Eles introduziram a propriedade de que, para cada nó, as alturas de suas subárvores esquerda e direita poderiam diferir em no máximo um.

Estruturas balanceadas são preferíveis às árvores binárias simples em aplicações que exigem desempenho previsível e eficiente para operações de busca, inserção e remoção.

Parte 3:

Descrição do experimento

O experimento consistiu na implementação de uma árvore AVL em linguagem C, com inserção, remoção e balanceamento automático por meio de rotações.

O código utiliza estruturas dinâmicas (ponteiros) e funções recursivas para garantir que a diferença de altura entre as subárvores esquerda e direita de cada nó seja, no máximo, 1.

Sequência de inserções:

30, 20, 10, 25, 28, 27, 40, 50, 45, 43

Sequência de remoções:

10, 27, 25, 43, 45, 50, 28, 20

Nó afetado	Tipo de rotação	Momento
30	RR (simples à direita)	Inserção do 10
30	LR (dupla à direita)	Inserção do 28
20	RL (dupla à esquerda)	Inserção do 27
30	LL(simples à esquerda)	Inserção do 50
28	LL (simples à esquerda)	Inserção do 45
50	RR (simples à direita)	Inserção do 43
25	LL(simples à esquerda)	Remoção 10
40	RR(simples à direita)	Remoção 50

Discussão

As rotações são o mecanismo principal que mantém o balanceamento da árvore AVL. Elas corrigem desequilíbrios que ocorrem após inserções ou remoções, garantindo que a diferença de alturas entre as subárvores esquerda e direita de qualquer nó nunca ultrapasse 1.

Sem o uso dessas rotações, a árvore se comportaria como uma árvore binária de busca comum (ABB).

Em inserções ordenadas, a árvore ficaria totalmente desbalanceada, assumindo o formato de uma lista encadeada. Nesse caso, as operações de busca, inserção e remoção deixariam de ser $O(\log n)$ e passariam a ser $O(n)$, prejudicando o desempenho.

Cada rotação é uma operação de custo constante $O(1)$, pois envolve apenas a troca de ponteiros entre nós.

Mesmo com o pequeno custo adicional de cálculo das alturas, a árvore AVL mantém a complexidade $O(\log n)$ para todas as operações, o que a torna ideal em contextos que exigem alto desempenho, como bancos de dados, compiladores e sistemas de indexação.

Conclusão

Durante o experimento, foi possível compreender o funcionamento e a importância dos quatro tipos de rotações no balanceamento da árvore AVL. Além disso, tornou-se visível a diferença entre o comportamento da AVL e de uma ABB comum.

Dentre os obstáculos encontrados, a dificuldade de compreensão das rotações duplas se destacou na realização da atividade.

Referências:

https://www.chessprogramming.org/Georgy_Adelson-Velsky#:~:text=was%20a%20Soviet%20and%20Israeli,Russian%20chess%20master%20Alexander%20Bitman.

Slides aula 09 - Estrutura de dados 2